第三章
栈和队列

【课前思考】

1. 什么是线性结构？

简单地说，线性结构是一个数据元素的序列。

2. 你见过餐馆中一叠一叠的盘子吗？如果它们是按 *1, 2, …, n* 的次序往上叠的，使用时候的次序什么？

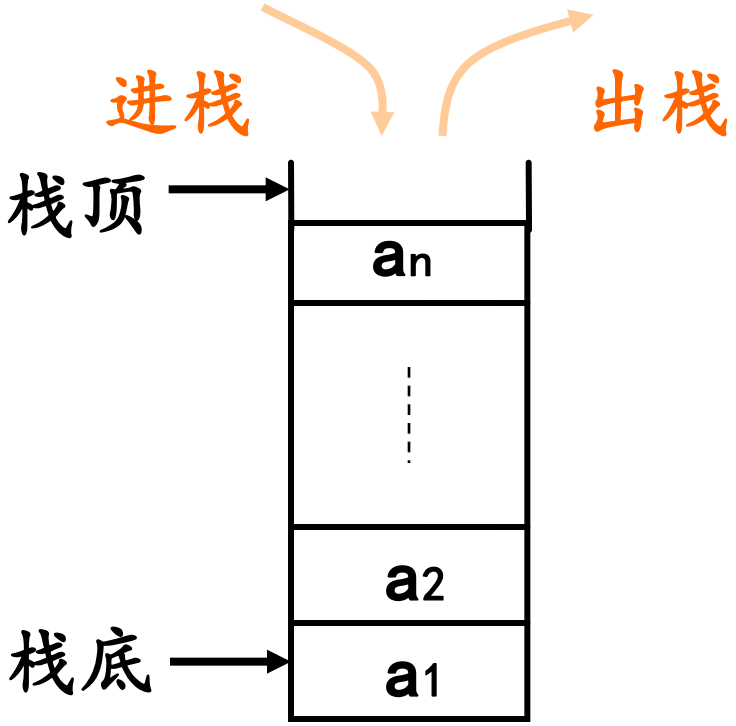
必然是依从上往下的次序，即n, …, 2, 1。遵循的是“后进先出”的规律，这就是"栈"的结构特点。

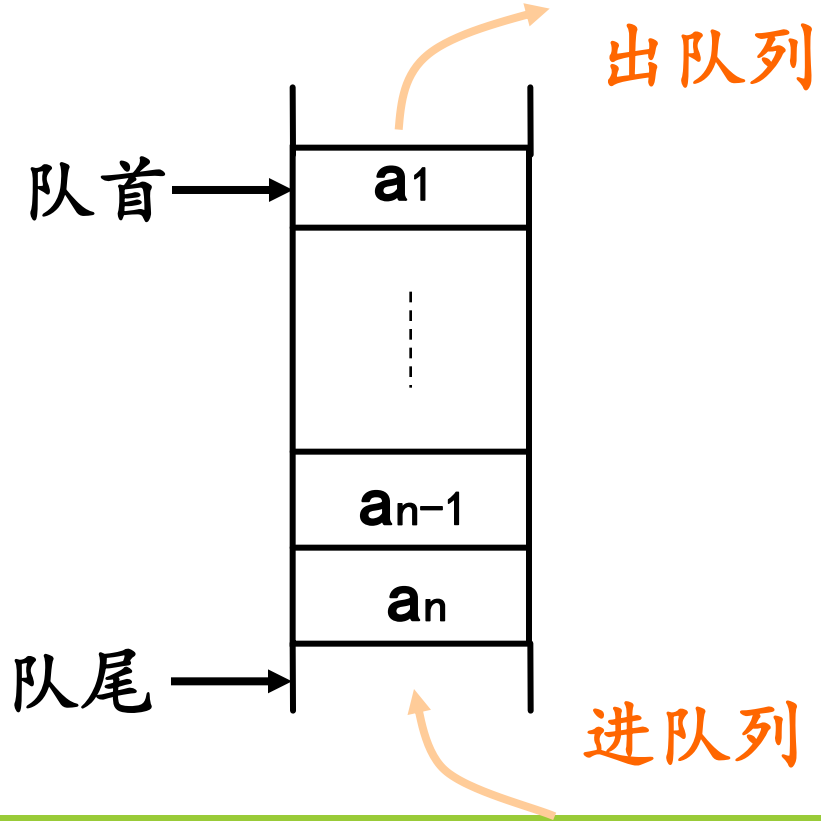
3. 在日常生活中，为了维持正常的社会秩序而出现的常见现象是什么？

"排队"。程序中，模拟排队的数据结构是"队列"。

什么是栈？什么是队列？

1. 栈和队列都是线性表
2. 但是其操作是受限的





通常称，栈和队列是限定**插入和删除只能**在表的“**端点**”进行的线性表。

线性表	**栈**	**队列**
Insert(L, **i**, x)	Insert(S, **n+1**, x)	Insert(Q, **n+1**, x)
1≤i≤n+1		
Delete(L, **i**)	Delete(S, **n**)	Delete(Q, **1**)
1≤i≤n		

3.1 栈的类型定义

3.2 栈类型的实现

3.3 栈的应用举例

3.4 队列的类型定义

3.5 队列类型的实现

栈（Stack）

- 栈 —— 仅在表的一端插、删的线性表

◆ 特点——先进后出或后进先出

设栈 $s=(a_1, a_2, \ldots, a_i, \ldots, a_n)$，

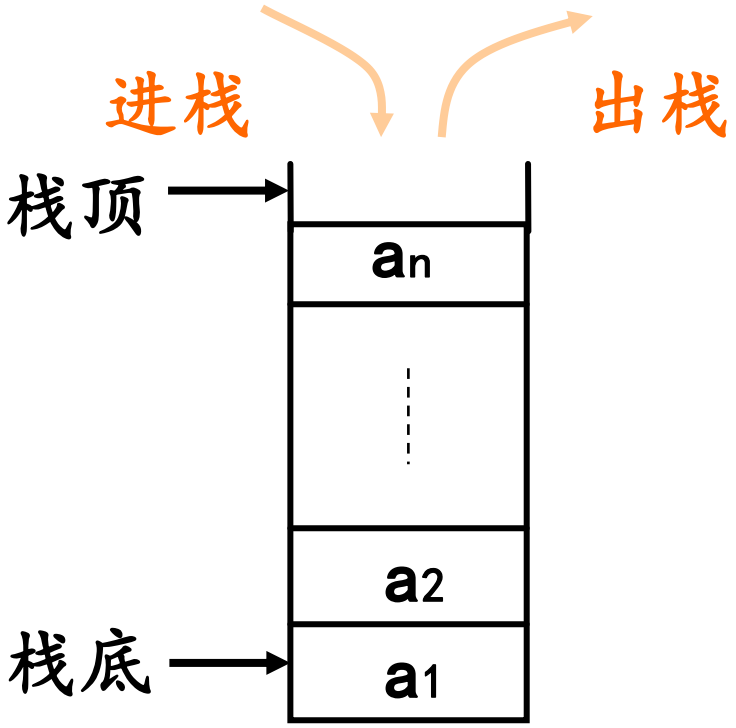
其中a_1是栈底元素， a_n是栈顶元素。

◆ 栈顶（top）—— 插删的一端

约定top始终指向新数据元素将存放的位置。

◆ 栈底（bottom）—— 不允许插入和删除的一端。

栈（Stack）



3.1 栈的类型定义

ADT Stack {

数据对象：

$D=\{ a_i \mid a_i \in ElemSet,\ i=1,2,...,n,\ n\geq 0 \}$

数据关系：

$R1=\{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D,\ i=2,...,n \}$

约定a_n 端为栈顶，a_1 端为栈底。

基本操作：

} ADT Stack

InitStack(&S)

DestroyStack(&S)

StackLength(S)

StackEmpty(S)

GetTop(S, &e)

ClearStack(&S)

Push(&S, e)

Pop(&S, &e)

StackTravers(S, visit())

- InitStack(&S)

操作结果：构造一个空栈 S。

- DestroyStack(&S)

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

- StackEmpty(S)

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 TRUE，否则 FALE。

- StackLength(S)

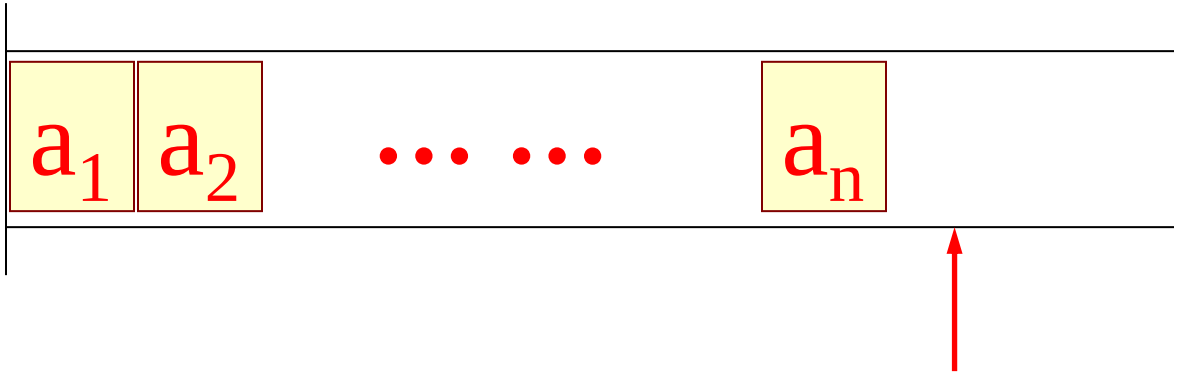
初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

- GetTop(S, &e)

初始条件：栈 S 已存在且非空

操作结果：用 e 返回 S 的栈顶元素。



- ClearStack(&S)

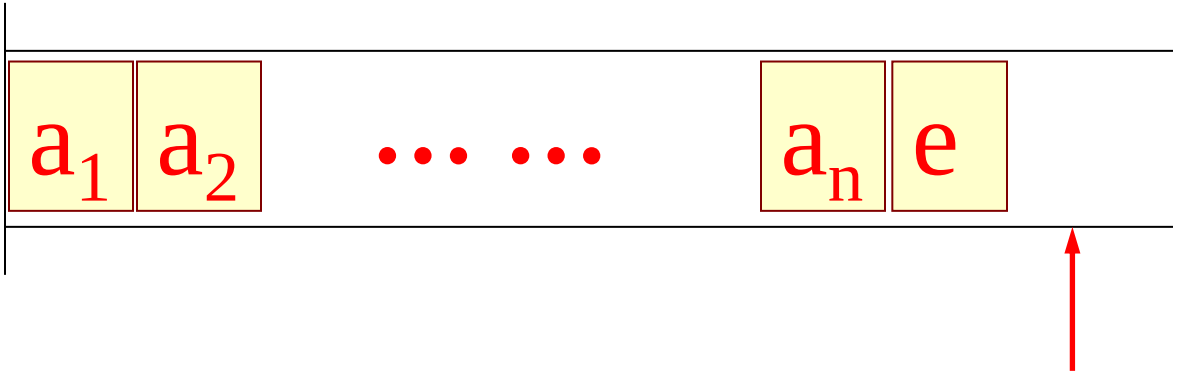
初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

- Push(&S, e)

初始条件：栈 S 已存在。

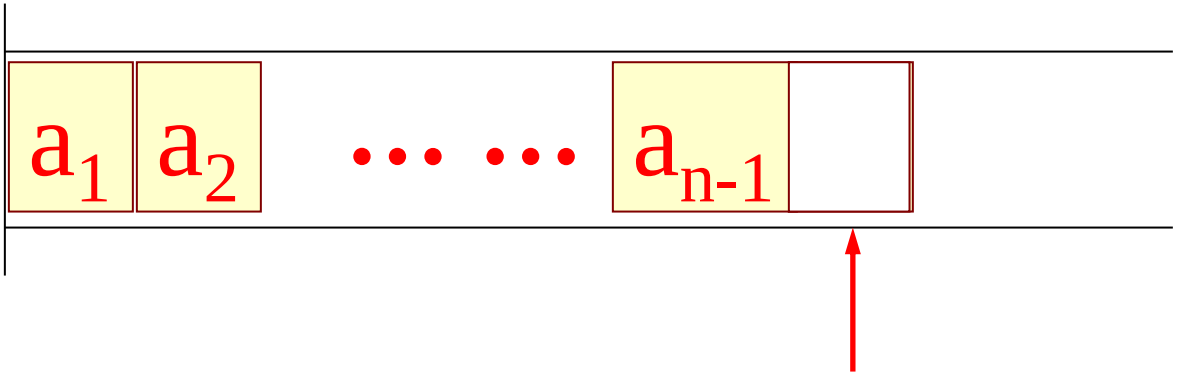
操作结果：插入 e 为新的栈顶元素。



- Pop(&S, &e)

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。



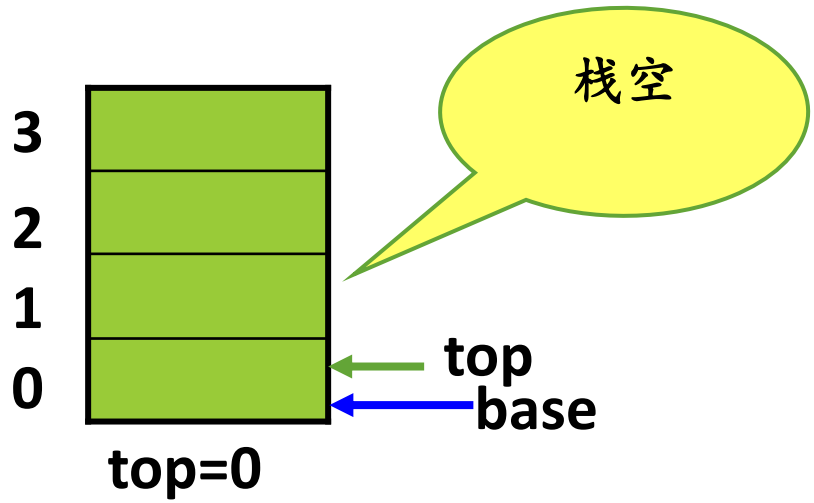
3.2 栈类型的实现

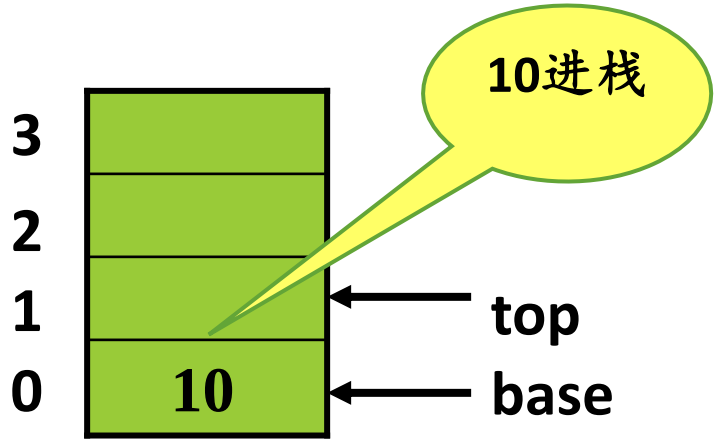
顺序栈

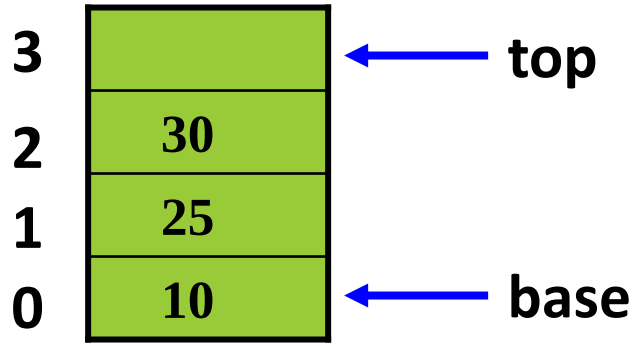
链 栈

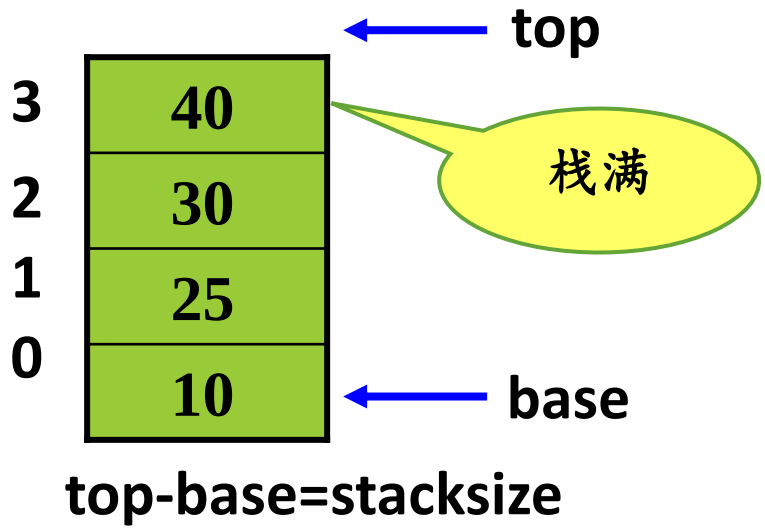
类似于线性表的顺序映象实现，
指向表尾的指针可以作为栈顶指针。

```
//----- 栈的顺序存储表示 -----
  #define  STACK_INIT_SIZE  100;
  typedef struct {
    SElemType  *base;
    SElemType  *top;      //栈顶指针
    int  stacksize;
  } SqStack;
```


栈空
3
2
1
0
top
base
top=0


10进栈
3
2
1
0
10
top
base


3
2
1
0
30
25
10
top
base


top
3
2
1
0
40
30
25
10
栈满
base
top-base=stacksize

```
Status InitStack (SqStack &S)
{// 构造一个空顺序栈 S
   S.base =(SElemType *)malloc(…);
   if (!S.base) exit (OVERFLOW); //存储分配失败
   S.top = S.base;
   S.stacksize =STACK_INIT_SIZE;
   return OK;
}  // InitStack
```

```
Status GetTop (SqStack S,SElemType &e)
{// 若栈不空，则用e返回S的栈顶元素，并返回OK，否
//则返回ERROR

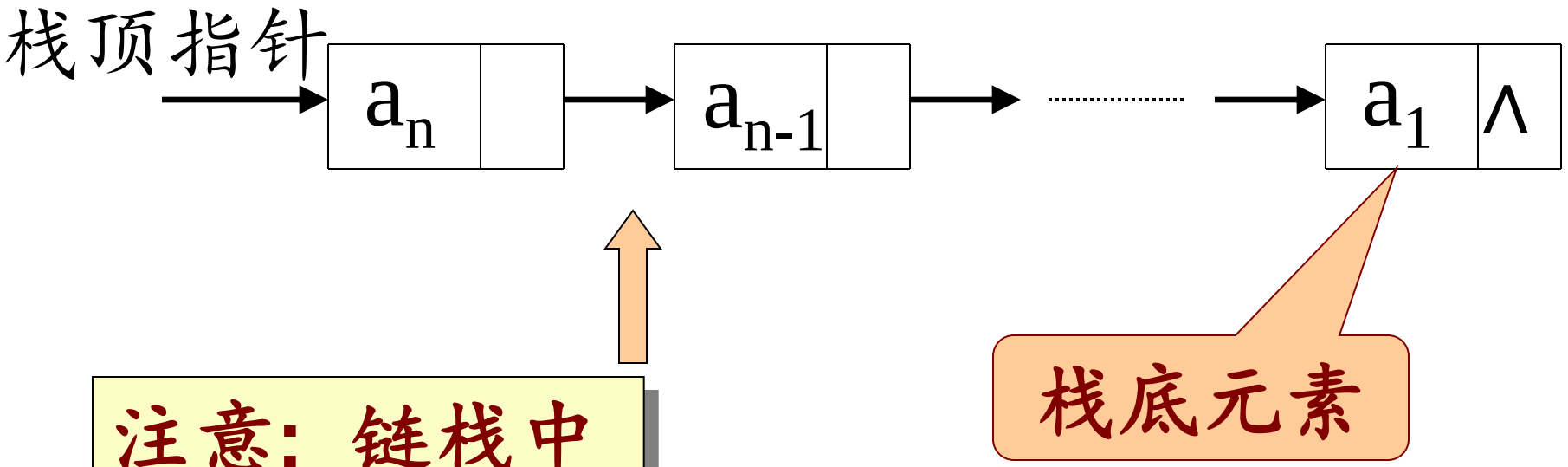
   if(S.base==S.top) return ERROR;
   e = *(S.top-1);
   return OK;

}//GetTop
```

```
Status Push (SqStack &S, SElemType e)
{ // 若栈不满，则将 e 插入栈顶
  if (S.top - S.base >= S.stacksize)  //栈满
  { //追加存储空间用realloc函数 }
  *S.top++ = e;
  return OK;
}//Push
```

```
Status Pop (SqStack &S, SElemType &e) {
    // 若栈不空，则删除S的栈顶元素，
    // 用 e 返回其值，并返回OK;
    // 否则返回ERROR
   if (S.top == S.base) return ERROR;
   e = *--S.top;
   return OK;
}//Pop
```

链栈



3.3 栈的应用举例

例一、 数制转换

例二、 括号匹配的检验

例一、数制转换

算法基于原理：

$$N = (N \text{ div } d) \times d + N \bmod d$$

例：$(4275)_8 = 4\times8^3+2\times8^2+7\times8^1+5\times8^0$

例如： $(1348)_{10} = (2504)_8$，其运算过程如下：

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

计算顺序 ↓

↑ 输出顺序

```
void conversion ()
{   InitStack(S);
    scanf ("%d",&N);
    while (N)
    {   Push(S, N % 8);  N = N/8;   }

    while (!StackEmpty(S))
    {  Pop(S,e);      printf ( "%d", e ); }
    DestroyStack(S);
} // conversion
```

例二、括号匹配的检验

假设在表达式中

（[]（））或[（[ ] [ ]）]

等为正确的格式，

[（]）或（[（））或(( )])

均为不正确的格式。

则 检验括号是否匹配的方法可用

“期待的急迫程度”这个概念来描述。

例如：考虑下列括号序列：

[([ ] [ ])]

1 2 3 4 5 6 7 8

分析可能出现的**不匹配**的情况：

到来的右括弧**非是所“期待”**的；

- 到来的是“不速之客”；
- 直到结束，也没有到来所“期待”的括弧；

算法的设计思想：

1）凡出现**左括弧**，则**进栈**；

2）凡出现**右括弧**，首先检查栈是否空，

若**栈空**，则表明该“**右括弧**”多余，

否则**和栈顶元素**比较，

若**相匹配**，则“**左括弧出栈**”

否则表明**不匹配**。

3）**表达式**检验**结束时**，

若**栈空**，则表明表达式中**匹配正确**，

否则表明“**左括弧**”有余。

```
Status matching(string& exp) {
  int state = 1; InitStack(S);
  while (i<=Length(exp) && state) {
    switch of exp[i] {
      case "(":{Push(S,exp[i]); i++; break;}
      case ")": {
        if( ! StackEmpty(S)&&GetTop(S)=="("  )
            {Pop(S,e);  i++;}
        else {state = 0;}
        break;  }
    … …  }
  if (StackEmpty(S)&&state) return OK; }
```

第四次作业：

借助栈(使用栈的基本运算)来实现单链表的逆置运算。

Void Invert(LinkList &L)

3.4 队列的类型定义



- 定义
  - 队列是只允许在一端删除，在另一端插入的线性表
  - 允许删除的一端叫做队头(*front*)，允许插入的一端叫做队尾(*rear*)。
- 特性
  - 先进先出(*FIFO, First In First Out*)

3.4 队列的类型定义

ADT Queue {

数据对象：

$$D=\{a_i \mid a_i \in ElemSet,\ i=1,2,...,n,\ n\geq 0\}$$

数据关系：

$$R1=\{ <a_{i-1},a_i> \mid a_{i-1},\ a_i \in D,\ i=2,...,n\}$$

约定其中 a_1 端为队列头，a_n 端为队列尾

基本操作：

} **ADT** Queue

队列的基本操作：

InitQueue(&Q)　　**DestroyQueue(&Q)**

QueueEmpty(Q)　　**QueueLength(Q)**

GetHead(Q, &e)　　**ClearQueue(&Q)**

EnQueue(&Q, e)　　**DeQueue(&Q, &e)**

QueueTravers(Q, visit())

InitQueue(&Q)

操作结果：构造一个空队列Q。

DestroyQueue(&Q)

初始条件：队列Q已存在。

操作结果：队列Q被销毁，不再存在。

QueueEmpty(Q)

初始条件：队列Q已存在。

操作结果：若Q为空队列，则返回TRUE，否则返回FALSE。

QueueLength(Q)

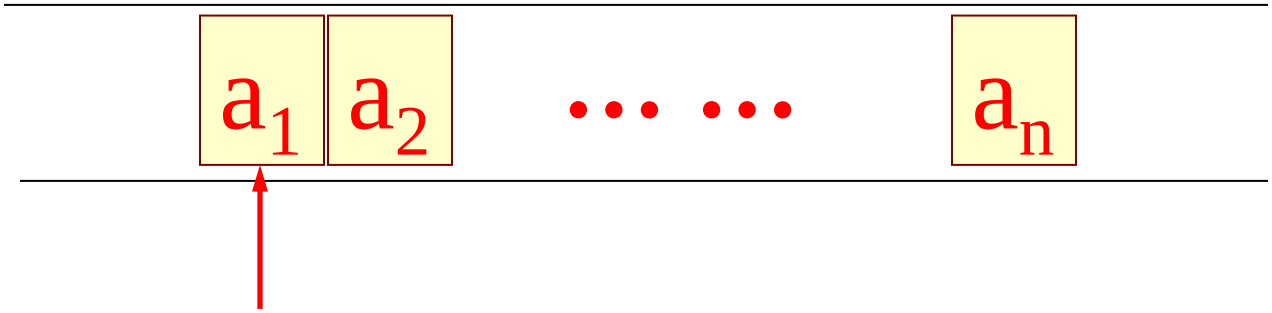
初始条件： 队列Q已存在。

操作结果： 返回Q的元素个数，即队列的长度。

GetHead(Q, &e)

初始条件： Q为非空队列。

操作结果： 用e返回Q的队头元素。



ClearQueue(&Q)

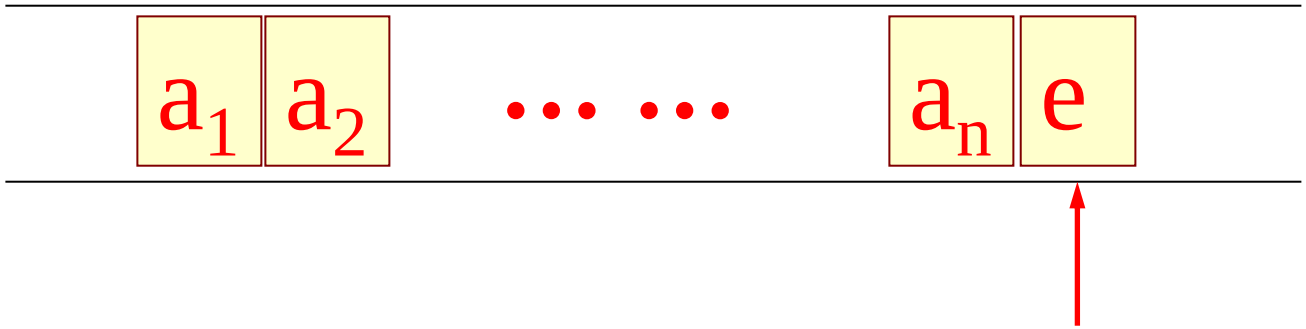
初始条件：队列Q已存在。

操作结果：将Q清为空队列。

EnQueue(&Q, e)

初始条件： 队列Q已存在。

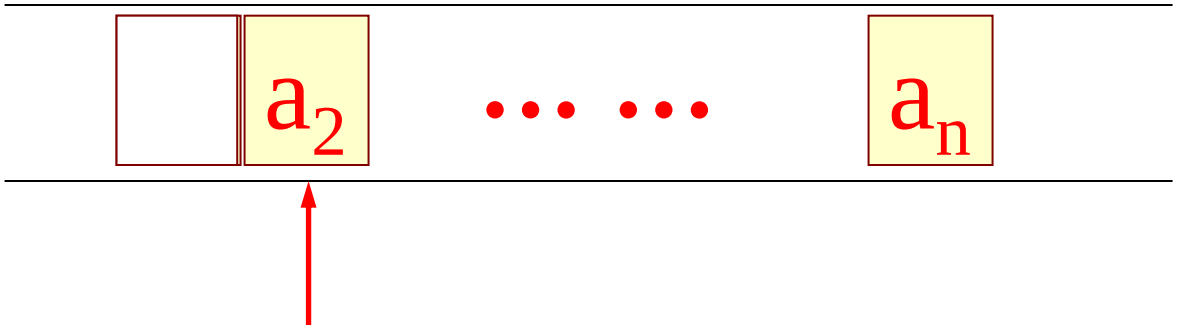
操作结果： 插入元素e为Q的新的队尾元素。



DeQueue(&Q, &e)

初始条件： Q为非空队列。

操作结果： 删除Q的队头元素，并用e返回其值。



3.5 队列类型的实现

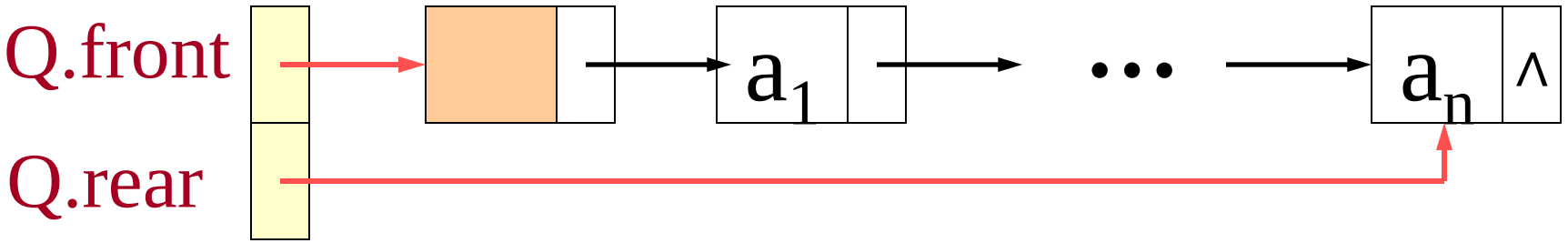
链队列——链式映象

循环队列——顺序映象

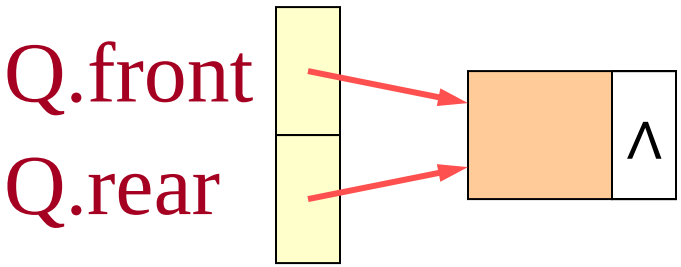
链队列——链式映象

```
typedef struct QNode {// 结点类型
  QElemType      data;
  struct QNode  *next;
} QNode, *QueuePtr;
```

```
typedef struct { // 链队列类型
    QueuePtr  front;  // 队头指针
    QueuePtr  rear;   // 队尾指针
} LinkQueue;
```

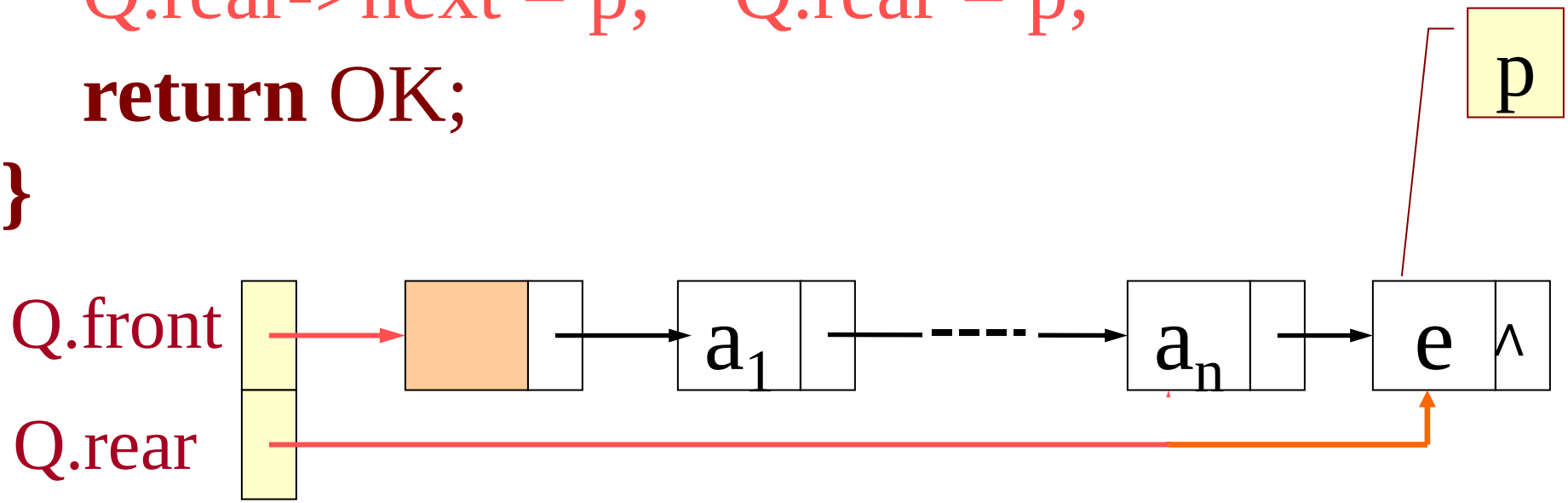


空队列



```
Status InitQueue (LinkQueue &Q) {
  // 构造一个空队列Q
  Q.front = Q.rear = new  QNode;
  if (!Q.front) exit (OVERFLOW); //存储分配失败
  Q.front->next = NULL;
  return OK;
}
```

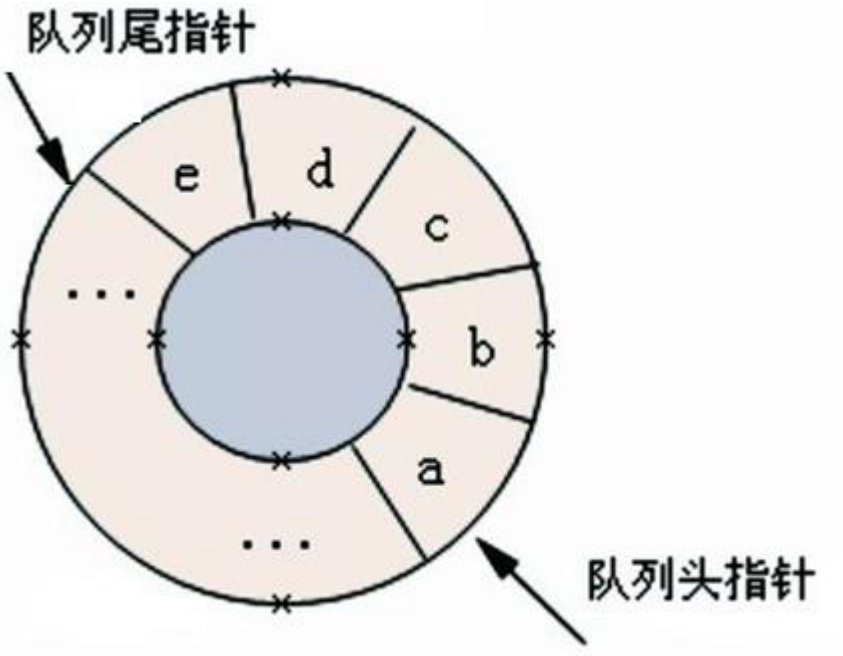
```
Status EnQueue (LinkQueue &Q, QElemType e) {
  // 插入元素e为Q的新的队尾元素
   p =  new QNode;
   if (!p)  exit (OVERFLOW);   //存储分配失败
   p->data = e;   p->next = NULL;
   Q.rear->next = p;    Q.rear = p;
   return OK;
}
```



```
Status DeQueue (LinkQueue &Q, QElemType &e) {
  // 若队列不空，则删除Q的队头元素，
  //用 e 返回其值，并返回OK；否则返回ERROR
   if (Q.front == Q.rear)    return ERROR;
   p = Q.front->next;   e = p->data;
   Q.front->next = p->next;
   if (Q.rear == p)  Q.rear = Q.front;  //空队列的处理
   delete (p);       return OK;
}
```

循环队列——顺序映象

为充分利用向量空间，克服"假溢出"现象的方法是：将向量空间想象为一个首尾相接的圆环，并称这种向量为循环向量。存储在其中的队列称为循环队列（Circular Queue）。



循环队列——顺序映象

为了方便起见，约定：初始化建空队时，令front=rear=0,

当队空时：front=rear

当队满时：front=rear 亦成立

因此只凭等式front=rear无法判断队空还是队满。

循环队列——顺序映象

有两种方法处理上述问题：

（1）另设一个标志位以区别队列是空还是满。

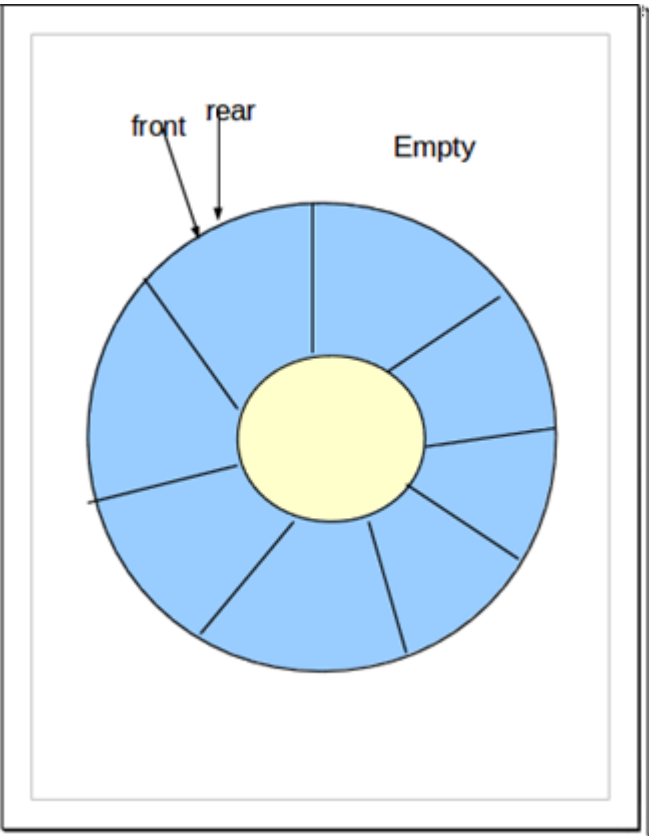
（2）少用一个元素空间，约定以“队列头指针front在队尾指针rear的下一个位置上”作为队列“满”状态的标志。即：

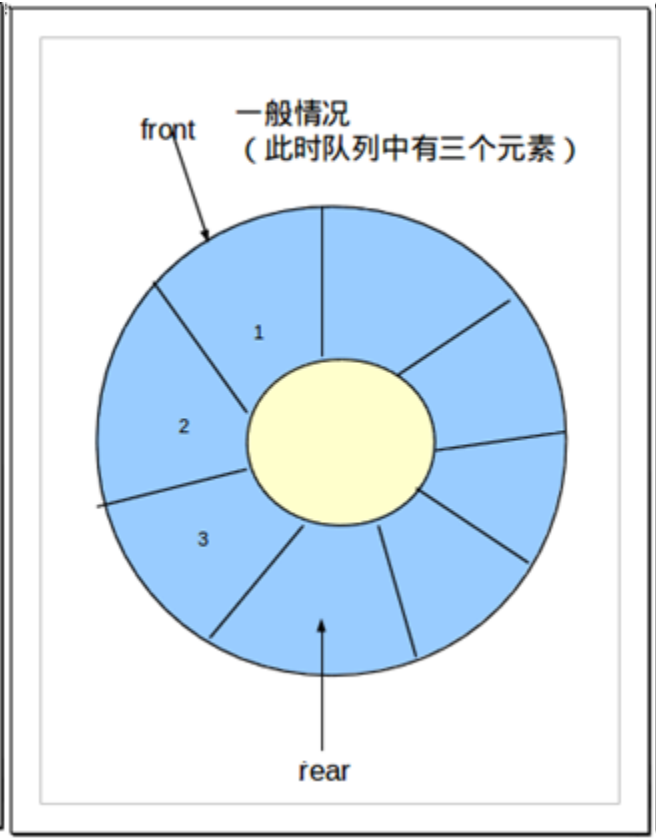
队空时： front=rear

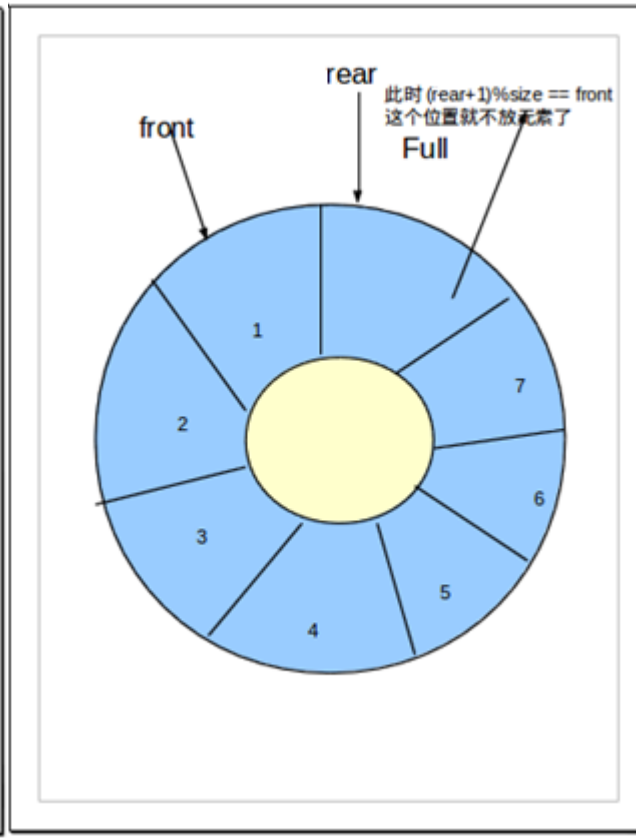
队满时： (rear+1)%maxsize=front

front指向队首元素，rear指向队尾元素的下一个元素。

循环队列——顺序映象


front
rear
Empty


front
一般情况
（此时队列中有三个元素）
1
2
3
rear


rear
此时 (rear+1)%size == front
这个位置就不放元素了
front
Full
1
2
3
4
5
6
7

循环队列——顺序映象

```
typedef struct {
    QElemType  *base;  // 动态分配存储空间
    int  front;     // 头指针，若队列不空，
                    //  指向队列头元素
    int  rear;      // 尾指针，若队列不空，指向
                    // 队列尾元素 的下一个位置
    int  queuesize;
} SqQueue;
```

```
Status InitQueue (SqQueue &Q, int maxsize) {
  // 构造一个最大存储空间为 maxsize 的
  // 空循环队列 Q
   Q.base = new ElemType[maxsize];
   if (!Q.base) exit (OVERFLOW);
   Q.queuesize = maxsize;
   Q.front = Q.rear = 0;
    return OK;
}
```

```
Status EnQueue (SqQueue &Q, ElemType e)
{  // 插入元素e为Q的新的队尾元素
   if ((Q.rear+1) % Q.queuesize == Q.front)
      return ERROR; //队列满
   Q.base[Q.rear] = e;
   Q.rear = (Q.rear+1) % Q.queuesize;
   return OK;
}
```

```
Status DeQueue (SqQueue &Q, ElemType &e)
{ // 若队列不空，则删除Q的队头元素，
  // 用e返回其值，并返回OK；否则返回ERROR
   if (Q.front == Q.rear)  return ERROR;
   e = Q.base[Q.front];
   Q.front = (Q.front+1) % Q.queuesize;
   return OK;
}
```

队列应用示例

1. 计算 n 行杨辉三角的值

二项式系数值（杨辉三角）

第 1 行　　1　1

第 2 行　　1　2　1

第 3 行　　1　3　3　1

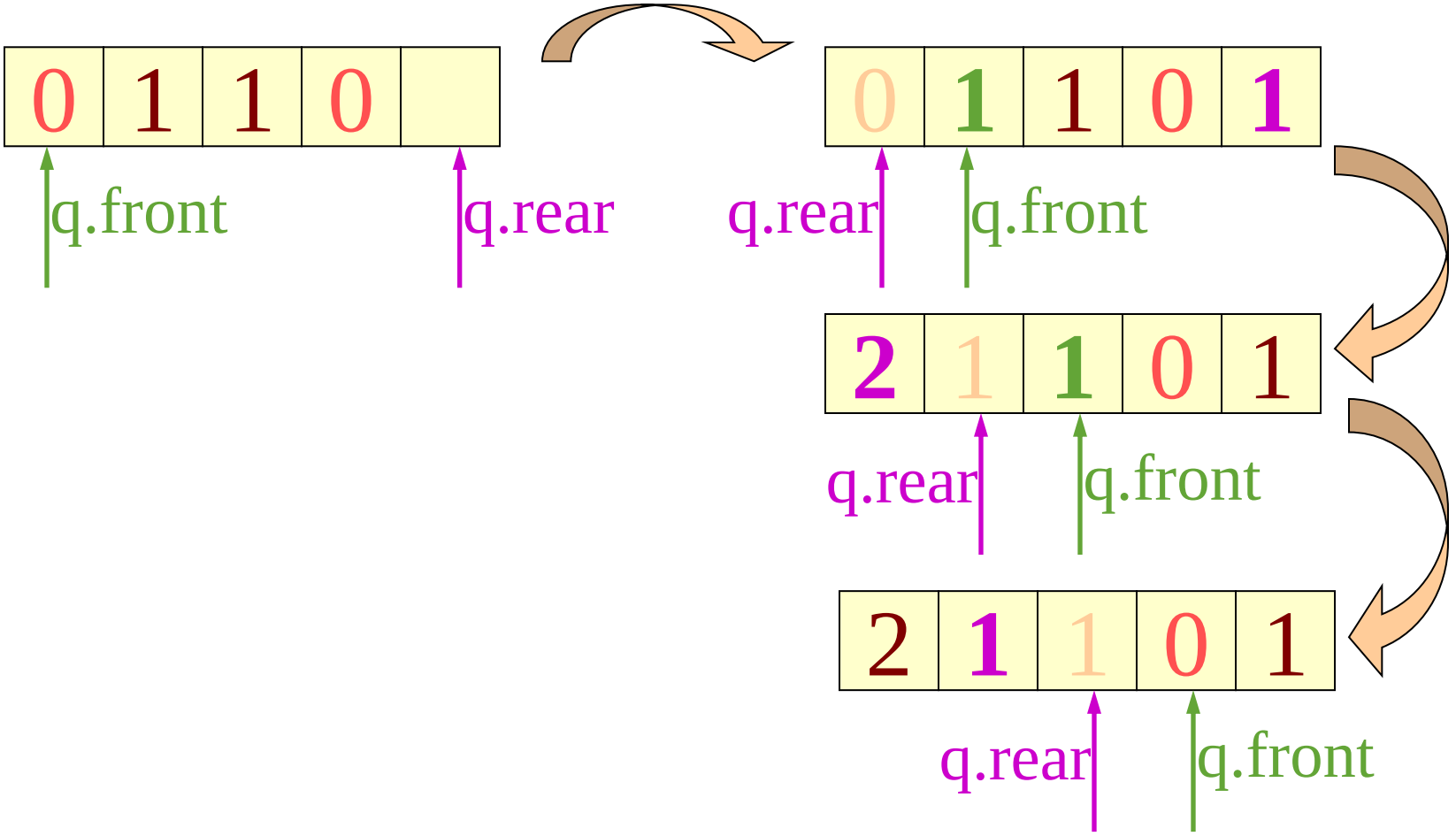
第 4 行　　1　4　6　4　1

设第 i-1行的值：(a[0]=0) a[1]..a[i] (a[i+1]=0)

则第 i 行的值：b[j] = a[j-1]+a[j], j=1,2,…,i+1

利用循环队列计算二项式的过程：

假设只计算2行，队列的最大容量为 5。



```
EnQueue(Q,0); /*第j行的结束符*/
do {
  DeQueue(Q, s);
  GetHead(Q, e);
  if (e!=0) printf (“%d”, e);
  EnQueue(Q, s+e);
} while (e!=0);
```

```
void YangHui(int n)
{ SqQueue q; InitQueue(q);
  EnQueue(q,0); EnQueue(q,1); EnQueue(q,1);
  for(j=1;j<=n;j++)
   {   EnQueue(q,0); /*第j行的结束符*/
       do{
        DeQueue(q,s); GetHead(q,t);
        if(t) printf("%d\t",t); /*非0输出，否则换行*/
        else printf("\n");
        EnQueue(q,s+t);
       }while(t!=0); /*遇到结束符结束循环*/
   }
   DeQueue(q);
}
```

本章小结

1. 掌握栈和队列类型的特点，并能在相应的应用问题中正确选用它们。

2. 熟练掌握栈类型的两种实现方法，特别应注意栈满和栈空的条件以及它们的描述方法。

3. 熟练掌握循环队列和链队列的基本操作实现算法，特别注意队满和队空的描述方法。

第五次作业：

对一个循环队列，实现操作输出每个元素.

Status OutputQueue(SqQueue Q)